

ĐẠI HỌC QUỐC GIA HÀ NỘI TRƯỜNG QUỐC TẾ



BÁO CÁO BÀI TẬP CUỐI KÌ
MÔN CƠ SỞ DỮ LIỆU

THIẾT KẾ ERD, CƠ SỞ DỮ LIỆU, CÁC CÂU LỆNH SQL

Hệ thống quản lý BrightPath Language Center

Lớp: ISV201604

Nhóm thực hiện: Nhóm 10

Thành viên: Nguyễn Tiến Sơn – 25070326

Đặng Cẩm Tú – 25070313

Phạm Hà My – 25070270

Doãn Thị Thu – 25070280

Giảng viên hướng dẫn: TS. Vũ Minh Đức

MỤC LỤC

MỤC LỤC.....	2
1. GIỚI THIỆU BÀI TOÁN	4
1.1. Mục tiêu	4
1.2. Phạm vi.....	4
1.3. Giả định và ngoài phạm vi	4
1.4. Yêu cầu dữ liệu và quy tắc nghiệp vụ	5
1.4.1. Danh sách quy tắc nghiệp vụ (BR01–BR15)	5
1.4.2. Bảng mapping quy tắc → ERD / constraint / enforcement layer	7
1.5. Change log (Cập nhật cuối kỳ).....	7
2. THIẾT KẾ ERD, RELATIONS VÀ CHUẨN HÓA	7
2.1. Mô tả entity	8
2.2. Sơ đồ ERD	8
2.3. Diễn giải cardinality và optionality	9
2.4. Relation schemas.....	11
2.5. Primary key, foreign key, alternate key	12
2.6. Data dictionary rút gọn.....	13
2.7. Mapping cần giải thích.....	13
2.8. Relation ban đầu để phân tích	14
2.9. Các phụ thuộc hàm (Functional Dependencies).....	14
2.10. Phân tích 1NF, 2NF, 3NF và decomposition	15
2.11. Ghi chú về denormalization	16
3. TRIỂN KHAI SCHEMA VÀ DỮ LIỆU.....	16
3.1. Quy ước đặt tên và lựa chọn kiểu dữ liệu.....	16
3.2. Thứ tự tạo bảng	16
3.3. CREATE DATABASE / CREATE TABLE.....	16
3.4. CREATE TABLE — bảng giao dịch và bảng kết hợp	18
3.5. DỮ LIỆU MẪU (Seed Data)	20
4. SQL QUERY PACK.....	22
Hệ thống sử dụng 8 câu truy vấn để giải quyết nghiệp vụ:	22
5. VIEWS VÀ STORED PROGRAMS	24
5.1. Hệ thống Báo cáo bằng View.....	24
5.2. Function đếm số buổi học	25
5.3. Procedure cấp chứng chỉ (Áp dụng TRANSACTION).....	26

6. TRIGGER VÀ EVENT THEO PHẠM VI	26
Trigger cho các rule không thể enforce bằng CREATE TABLE cơ bản	26
7. INDEX VÀ EXPLAIN.....	28
7.1. Indexes và query mẫu.....	28
7.2. Explain	29
8. ADMINISTRATION AND BACKUP.....	29
8.1. Phân quyền truy cập (Least Privilege)	29
8.2. Kế hoạch Backup/Restore cục bộ.....	29
9. KIỂM THỬ VÀ KẾT LUẬN.....	30
9.1. Sample data	30
9.2. Positive tests.....	31
9.3. Negative tests	31
9.4. Query đối chiếu và kết quả.....	31
9.5. Kết luận	32

1. GIỚI THIỆU BÀI TOÁN

1.1. Mục tiêu

BrightPath Language Center là một trung tâm đào tạo ngoại ngữ với hàng trăm học viên theo học các lớp tiếng Anh, tiếng Nhật và tiếng Hàn. Mục tiêu của báo cáo là thiết kế và triển khai một cơ sở dữ liệu quan hệ phục vụ bốn nhóm nghiệp vụ cốt lõi của trung tâm:

1. Quản lý lịch lớp học cố định (cấp độ ngôn ngữ, ngày trong tuần, giờ học, phòng học).
2. Quản lý người phụ trách giảng dạy (trợ giảng/giảng viên), bao gồm việc phân công phụ trách chính cho từng lớp và việc hỗ trợ giảng dạy theo từng buổi cụ thể.
3. Ghi nhận điểm danh của học viên theo từng buổi học cụ thể, không ràng buộc học viên vào một lớp cố định duy nhất.
4. Quản lý hệ thống chứng chỉ nội bộ và lịch sử đạt chứng chỉ của từng học viên.

Báo cáo trình bày đầy đủ chuỗi lập luận: từ yêu cầu nghiệp vụ, quy tắc nghiệp vụ, thiết kế ERD, mô hình quan hệ, phân tích phụ thuộc hàm/chuẩn hóa, cho đến script SQL triển khai trên MySQL 8+ và các trường hợp kiểm thử minh họa.

1.2. Phạm vi

Phạm vi thiết kế bao gồm các đối tượng và quy trình được nêu trong đề bài gốc:

- Lớp học cố định theo lịch (level, thứ, giờ, phòng) và các buổi học cụ thể phát sinh theo thời gian.
- Học viên, bao gồm cả các học viên đồng thời là người phụ trách giảng dạy (trợ giảng/giảng viên).
- Việc phân công người phụ trách chính cho lớp, và việc hỗ trợ giảng dạy theo từng buổi (vai trò lead/assistant).
- Điểm danh học viên theo từng buổi học cụ thể (không phải theo lớp cố định).
- Hệ thống chứng chỉ nội bộ, các yêu cầu của từng chứng chỉ, và lịch sử chứng chỉ học viên đã đạt được, bao gồm việc tự động cấp Starter Badge.

Ngoài phạm vi của báo cáo này: quản lý học phí/thanh toán, quản lý phòng học vật lý (sức chứa, thiết bị), hệ thống thông báo, và các module báo cáo thống kê nâng cao (dashboard, BI). Đây là các hướng mở rộng có thể bổ sung ở các phiên bản sau mà không phá vỡ mô hình dữ liệu hiện tại.

1.3. Giả định và ngoài phạm vi

Do đề bài không nêu chi tiết một số quy tắc, nhóm đưa ra các giả định sau, có ghi chú tác động thiết kế đi kèm:

- G1: Mỗi buổi học (CLASS_MEETING) chỉ diễn ra một lần tại một ngày cụ thể; không có khái niệm hủy buổi/dạy bù trong phạm vi báo cáo (có thể mở rộng bằng thuộc tính status ở phiên bản sau).
- G2: “Người phụ trách” (STAFF/INSTRUCTOR) trong đề bài được hiểu thống nhất là một vai trò chuyên biệt hóa (subtype) của STUDENT, vì đề bài khẳng định rõ “tất cả người phụ trách giảng dạy đều phải là học viên hoặc cựu học viên”.

- G3: Một học viên có thể vừa là INSTRUCTOR vừa tiếp tục là STUDENT thông thường (không loại trừ lẫn nhau); do đó quan hệ subtype là partial, không disjoint bắt buộc theo chiều thời gian (một người có thể ngừng là instructor nhưng dữ liệu lịch sử vẫn được giữ lại).
- G4: badgeColor của CERTIFICATE được lưu dưới dạng chuỗi mã màu hoặc tên màu (ví dụ: 'gold', '#FFD700'), không ràng buộc thành bảng danh mục màu riêng vì đề bài không yêu cầu.
- G5: Việc “tự động cấp Starter Badge cho học viên mới” là một business rule thuộc tầng ứng dụng/trigger, không thể (và không nên) enforce cứng bằng CHECK constraint thuần túy trong DDL.
- G6: Ràng buộc “mọi chứng chỉ phải có ít nhất một học viên từng đạt được” là ràng buộc tồn tại tối thiểu (participation constraint) khó biểu diễn trực tiếp bằng FK/CHECK ở schema rộng; được liệt kê riêng ở Chương 9 (báo cáo) / Chương 6 (SQL).
- CH01: Thêm bảng attendance_audit và AFTER DELETE trigger để lưu vết lịch sử xóa điểm danh.
- CH02: Bổ sung SQL Query Pack, Views, Procedures (có Transaction) và Event dọn dẹp hệ thống.
- CH03: Bổ sung phân quyền User Report và quy trình Backup hệ thống.

1.4. Yêu cầu dữ liệu và quy tắc nghiệp vụ

1.4.1. Danh sách quy tắc nghiệp vụ (BR01–BR15)

Phần này trích xuất các đối tượng, thông tin cần lưu, và quy tắc nghiệp vụ trực tiếp từ 12 yêu cầu của đề bài, sau đó chuẩn hóa thành các mã quy tắc (Business Rules – BR) có thể truy vết đến ERD, mô hình quan hệ và ràng buộc SQL tương ứng.

Mã	Quy tắc nghiệp vụ (trích từ đề bài)	Hệ quả thiết kế
BR01	Mỗi lớp học (CLASS) được mở cho một cấp độ ngôn ngữ, một ngày trong tuần, một giờ học, một phòng học cụ thể (yêu cầu 1).	CLASS là entity với thuộc tính level, dayOfWeek, timeSlot, room.
BR02	Mỗi học viên có mã học viên, họ tên, ngày sinh, ngày bắt đầu học (yêu cầu 2).	STUDENT(studentID PK, fullName, dateOfBirth, enrollmentDate).
BR03	Mỗi chứng chỉ nội bộ có tên, màu huy hiệu, mô tả năng lực (yêu cầu 3).	CERTIFICATE(certID PK, certName, badgeColor, description).
BR04	Mỗi lớp học phải có đúng một người phụ trách chính; một người phụ trách có thể phụ trách nhiều lớp; một số người phụ trách (tình nguyện) có thể chưa được phân công lớp nào (yêu cầu 4).	Quan hệ INSTRUCTOR 1 – N CLASS (bắt buộc phía CLASS, tùy chọn phía INSTRUCTOR); FK headInstructorID NOT NULL trên CLASS.
BR05	Học viên không nhất thiết theo lớp cố định hằng tuần; cần ghi nhận điểm danh theo từng buổi học cụ thể (yêu cầu 5).	Tạo CLASS_MEETING (buổi học cụ thể) và associative entity ATTENDANCE (STUDENT – CLASS_MEETING), không dùng quan hệ STUDENT – CLASS trực tiếp.
BR06	Mọi người phụ trách đều là học viên/cựu học viên, nhưng không phải mọi học viên đều là người phụ trách; cần lưu ngày bắt đầu hỗ trợ và trạng thái paid/volunteer (yêu cầu 6).	INSTRUCTOR là subtype (partial) của STUDENT; thuộc tính staffStartDate, status ENUM('paid','volunteer') đặt tại

Mã	Quy tắc nghiệp vụ (trích từ đề bài)	Hệ quả thiết kế
		INSTRUCTOR, không đặt ở STUDENT để tránh NULL thừa.
BR07	Một lớp có thể diễn ra nhiều buổi khác nhau theo thời gian; mỗi buổi lưu ngày diễn ra và gắn với đúng một lớp cố định (yêu cầu 7).	Quan hệ CLASS 1 – N CLASS_MEETING; CLASS_MEETING có FK classID NOT NULL.
BR08	Hầu hết chứng chỉ có nhiều yêu cầu; mỗi yêu cầu thuộc đúng một chứng chỉ; Starter Badge có thể không có yêu cầu, các chứng chỉ khác phải có ít nhất một yêu cầu (yêu cầu 8).	Quan hệ CERTIFICATE 1 – N REQUIREMENT (0..N cho Starter Badge, 1..N cho các chứng chỉ còn lại — ràng buộc có điều kiện, xem BR15).
BR09	Một buổi học có thể có nhiều học viên tham dự hoặc không có học viên nào; một học viên có thể tham dự nhiều buổi hoặc chưa tham dự buổi nào (yêu cầu 9).	ATTENDANCE là quan hệ N–M, participation tùy chọn (0..N) ở cả hai phía.
BR10	Trong một buổi học, ngoài người phụ trách chính của lớp, có thể có người hỗ trợ khác đến; mỗi buổi có thể có nhiều người phụ trách với vai trò lead/assistant (yêu cầu 10).	Associative entity MEETING_STAFF (CLASS_MEETING – INSTRUCTOR) với thuộc tính role ENUM('lead','assistant').
BR11	Học viên có thể đạt nhiều chứng chỉ theo thời gian; cần lưu toàn bộ lịch sử, mỗi chứng chỉ trao có ngày được trao (yêu cầu 11).	Associative entity STUDENT_CERTIFICATE (STUDENT – CERTIFICATE) với thuộc tính awardDate; PK phức hợp (studentID, certID) — không unique theo cặp vì mỗi chứng chỉ chỉ trao một lần cho một học viên.
BR12	Học viên mới tự động được cấp Starter Badge (yêu cầu 12, phần 1).	Rule thuộc tăng ứng dụng/trigger AFTER INSERT trên STUDENT → INSERT INTO STUDENT_CERTIFICATE.
BR13	Mọi chứng chỉ phải có ít nhất một học viên từng đạt được tại một thời điểm nào đó (yêu cầu 12, phần 2).	Ràng buộc tồn tại tối thiểu (participation ≥ 1) phía CERTIFICATE → STUDENT_CERTIFICATE; không enforce được bằng CHECK/FK cơ bản, cần kiểm tra ở tầng ứng dụng hoặc quy trình seed dữ liệu (xem Chương 9).
BR14	Vai trò lead/assistant và status paid/volunteer là các thuộc tính có miền giá trị rõ ràng.	Dùng CHECK/ENUM tương ứng: MEETING_STAFF.role, INSTRUCTOR.status.
BR15	Ràng buộc điều kiện: Starter Badge có thể có 0 yêu cầu, các chứng chỉ khác bắt buộc ≥ 1 yêu cầu.	Không thể hiện được bằng một CHECK constraint đơn giản trên REQUIREMENT (vì điều kiện phụ thuộc certName cụ thể và đếm số dòng liên quan); cần trigger hoặc kiểm tra ứng dụng khi thêm CERTIFICATE mới (xem Chương 9).

1.4.2. Bảng mapping quy tắc → ERD / constraint / enforcement layer

Mã BR	Thể hiện trên ERD	Constraint SQL	Enforcement khác
BR04	CLASS —(1)—crow's foot(0..N)— INSTRUCTOR	FK head_instructor_id NOT NULL	—
BR05/BR09	STUDENT =N–M= CLASS_MEETING qua ATTENDANCE	PK phức hợp (student_id, meeting_id)	—
BR06	Subtype INSTRUCTOR của STUDENT (partial)	FK instructor_id → student_id (1–1)	Ứng dụng kiểm tra student tồn tại trước khi thêm instructor
BR08/BR15	CERTIFICATE —(1)—crow's foot(0..N *)— REQUIREMENT	FK cert_id NOT NULL trên REQUIREMENT	Trigger/ứng dụng kiểm tra ≥1 requirement khi cert ≠ Starter Badge
BR10	CLASS_MEETING =N–M= INSTRUCTOR qua MEETING_STAFF	CHECK role IN (‘lead’,‘assistant’)	Ứng dụng đảm bảo mỗi buổi có tối đa 1 ‘lead’
BR11	STUDENT =N–M= CERTIFICATE qua STUDENT_CERTIFICATE	PK phức hợp (student_id, cert_id); award_date NOT NULL	—
BR12	(không thể hiện trực tiếp trên ERD — quy trình nghiệp vụ)	—	Trigger AFTER INSERT ON student
BR13	CERTIFICATE tham gia tối thiểu 1..N với STUDENT_CERTIFICATE	—	Kiểm tra ứng dụng khi tạo CERTIFICATE mới + seed data đủ ví dụ

Phân biệt các loại quy tắc (theo hướng dẫn mục 3.2)

- Rule cấu trúc (PK/FK/UNIQUE/NOT NULL/CHECK/bảng kết hợp): BR01–BR11, BR14.
- Rule theo trạng thái/quy trình (cần trigger/procedure/ứng dụng): BR12 (tự động cấp Starter Badge), BR15 (điều kiện số lượng yêu cầu theo loại chứng chỉ).
- Rule tồn tại tối thiểu khó enforce bằng DDL cơ bản: BR13 (mọi chứng chỉ phải từng được trao).

1.5. Change log (Cập nhật cuối kỳ)

- **CH01:** Thêm bảng attendance_audit và AFTER DELETE trigger để lưu vết lịch sử xóa điểm danh.
- **CH02:** Bổ sung SQL Query Pack, Views, Procedures (có Transaction) và Event dọn dẹp hệ thống.
- **CH03:** Bổ sung phân quyền User Report và quy trình Backup hệ thống.

2. THIẾT KẾ ERD, RELATIONS VÀ CHUẨN HÓA

2.1. Mô tả entity

Hệ thống gồm 9 thực thể: 2 entity thông thường có quan hệ subtype/supertype (STUDENT – INSTRUCTOR), 4 entity thông thường khác (CLASS, CLASS_MEETING, CERTIFICATE, REQUIREMENT), và 3 associative entity giải quyết các quan hệ N–M có thuộc tính riêng (ATTENDANCE, MEETING_STAFF, STUDENT_CERTIFICATE).

Entity	Loại	Lý do chọn / Vai trò nghiệp vụ	Khóa chính
STUDENT	Regular (supertype)	Đối tượng trung tâm của hệ thống; mọi học viên đều được cấp mã khi đăng ký (BR02).	studentID
INSTRUCTOR	Subtype (partial) của STUDENT	Người phụ trách giảng dạy — luôn là học viên/cựu học viên nhưng có thuộc tính riêng (staffStartDate, status) không áp dụng cho student thường (BR06).	studentID (FK/PK)
CLASS	Regular	Lớp học theo lịch cố định — có nhiều thuộc tính riêng (level, dayOfWeek, timeSlot, room) và một FK bắt buộc tới người phụ trách chính (BR01, BR04).	classID
CLASS_MEETING	Regular (sự kiện lặp)	Buổi học cụ thể phát sinh theo thời gian từ một CLASS; cần lưu ngày diễn ra riêng biệt (BR07).	meetingID
CERTIFICATE	Regular	Danh mục chứng chỉ nội bộ của trung tâm, có thuộc tính riêng (certName, badgeColor, description) (BR03).	certID
REQUIREMENT	Regular (phụ thuộc yếu)	Yêu cầu cụ thể của một chứng chỉ; luôn thuộc đúng một CERTIFICATE, không tồn tại độc lập (BR08).	reqID
ATTENDANCE	Associative (N–M)	Giải quyết quan hệ nhiều-nhiều STUDENT – CLASS_MEETING; bản thân không có thuộc tính mô tả ngoài khóa (điểm danh) (BR05, BR09).	(studentID, meetingID)
MEETING_STAFF	Associative (N–M) có thuộc tính	Giải quyết quan hệ nhiều-nhiều CLASS_MEETING – INSTRUCTOR; có thuộc tính riêng role (lead/assistant) (BR10).	(meetingID, instructorID)
STUDENT_CERTIFICATE	Associative (N–M) có thuộc tính	Giải quyết quan hệ nhiều-nhiều STUDENT – CERTIFICATE; có thuộc tính riêng awardDate, lưu lịch sử đạt chứng chỉ (BR11, BR12).	(studentID, certID)

Vì sao không tạo entity “STAFF_ROLE” hay “LANGUAGE” riêng: đề bài chỉ nêu level (cấp độ ngôn ngữ) như một thuộc tính mô tả của CLASS (ví dụ 'Intermediate English', 'Beginner Korean' – gộp cả ngôn ngữ và cấp độ trong một giá trị), không có thuộc tính hay quan hệ riêng nào khác gắn với *ngôn ngữ* như một đối tượng độc lập, nên nhóm giữ level là thuộc tính đơn của CLASS thay vì tách entity LANGUAGE — tránh tạo entity thừa khi chỉ cần thuộc tính (theo nguyên tắc ở Mục 3.1 của tài liệu hướng dẫn).

2.2. Sơ đồ ERD

2.3. Diễn giải cardinality và optionality

Entity	Loại	Lý do chọn / Vai trò nghiệp vụ	Khóa chính
STUDENT	Regular (supertype)	Đối tượng trung tâm của hệ thống; mọi học viên đều được cấp mã khi đăng ký (BR02).	studentID
INSTRUCTOR	Subtype (partial) của STUDENT	Người phụ trách giảng dạy — luôn là học viên/cựu học viên nhưng có thuộc tính riêng (staffStartDate, status) không áp dụng cho student thường (BR06).	studentID (FK/PK)
CLASS	Regular	Lớp học theo lịch cố định — có nhiều thuộc tính riêng (level, dayOfWeek, timeSlot, room) và một FK bắt buộc tới người phụ trách chính (BR01, BR04).	classID

Entity	Loại	Lý do chọn / Vai trò nghiệp vụ	Khóa chính
CLASS_MEETING	Regular (sự kiện lặp)	Buổi học cụ thể phát sinh theo thời gian từ một CLASS; cần lưu ngày diễn ra riêng biệt (BR07).	meetingID
CERTIFICATE	Regular	Danh mục chứng chỉ nội bộ của trung tâm, có thuộc tính riêng (certName, badgeColor, description) (BR03).	certID
REQUIREMENT	Regular (phụ thuộc yếu)	Yêu cầu cụ thể của một chứng chỉ; luôn thuộc đúng một CERTIFICATE, không tồn tại độc lập (BR08).	reqID
ATTENDANCE	Associative (N–M)	Giải quyết quan hệ nhiều-nhiều STUDENT – CLASS_MEETING; bản thân không có thuộc tính mô tả ngoài khóa (điểm danh) (BR05, BR09).	(studentID, meetingID)
MEETING_STAFF	Associative (N–M) có thuộc tính	Giải quyết quan hệ nhiều-nhiều CLASS_MEETING – INSTRUCTOR; có thuộc tính riêng role (lead/assistant) (BR10).	(meetingID, instructorID)
STUDENT_CERTIFICATE	Associative (N–M) có thuộc tính	Giải quyết quan hệ nhiều-nhiều STUDENT – CERTIFICATE; có thuộc tính riêng awardDate, lưu lịch sử đạt chứng chỉ (BR11, BR12).	(studentID, certID)

Vì sao không tạo entity “STAFF_ROLE” hay “LANGUAGE” riêng: đề bài chỉ nêu level (cấp độ ngôn ngữ) như một thuộc tính mô tả của CLASS (ví dụ 'Intermediate English', 'Beginner Korean' – gộp cả ngôn ngữ và cấp độ trong một giá trị), không có thuộc tính hay quan hệ riêng nào khác gắn với *ngôn ngữ* như một đối tượng độc lập, nên nhóm giữ level là thuộc tính đơn của CLASS thay vì tách entity LANGUAGE — tránh tạo entity thừa khi chỉ cần thuộc tính (theo nguyên tắc ở Mục 3.1 của tài liệu hướng dẫn).

Quan hệ	Diễn giải nghiệp vụ
STUDENT (1) — INSTRUCTOR (P, subtype)	Một STUDENT có thể (0..1) là một INSTRUCTOR; mỗi INSTRUCTOR tương ứng đúng 1 STUDENT. Subtype partial vì không phải học viên nào cũng là người phụ trách (BR06).
INSTRUCTOR (1) — CLASS (0..N)	Một INSTRUCTOR có thể phụ trách chính 0..N CLASS (người mới/tình nguyện có thể chưa phụ trách lớp nào); mỗi CLASS phải có đúng 1 INSTRUCTOR phụ trách chính (BR04).
CLASS (1) — CLASS_MEETING (0..N)	Một CLASS có thể có 0..N buổi học cụ thể theo thời gian; mỗi CLASS_MEETING thuộc đúng 1 CLASS (BR07).
STUDENT (1) — ATTENDANCE (0..N)	Một STUDENT có thể tham dự 0..N buổi (học viên mới có thể chưa tham dự buổi nào); mỗi dòng ATTENDANCE gắn với đúng 1 STUDENT (BR09).
CLASS_MEETING (1) — ATTENDANCE (0..N)	Một buổi học có thể có 0..N học viên tham dự (có thể không ai tham dự); mỗi dòng ATTENDANCE gắn với đúng 1 CLASS_MEETING (BR09).

Quan hệ	Diễn giải nghiệp vụ
CLASS_MEETING – INSTRUCTOR qua MEETING_STAFF	Một buổi học có 1..N người phụ trách (tối thiểu người phụ trách chính); một INSTRUCTOR có thể hỗ trợ 0..N buổi khác nhau (BR10).
CERTIFICATE (1) — REQUIREMENT (0..N)*	Một CERTIFICATE có 0..N REQUIREMENT (riêng Starter Badge có thể là 0); mỗi REQUIREMENT thuộc đúng 1 CERTIFICATE (BR08, BR15 — điều kiện đặc biệt xem Chương 9).
STUDENT – CERTIFICATE qua STUDENT_CERTIFICATE	Một STUDENT có thể đạt 0..N CERTIFICATE theo thời gian; một CERTIFICATE phải được trao cho tối thiểu 1..N STUDENT (BR11, BR13 — ràng buộc tồn tại tối thiểu xem Chương 9).

Lưu ý (theo hướng dẫn Mục 5)

Foreign key là khái niệm của logical/physical model. Trên ERD ở Hình 1, FK được liệt kê tường minh trong khối thuộc tính của entity (ví dụ headInstructorID trên CLASS) để tiện đối chiếu, đồng thời quan hệ crow's foot vẫn được vẽ song song để thể hiện cardinality/participation. Cách trình bày này được giữ nhất quán xuyên suốt Chương 2.

2.4. Relation schemas

```

STUDENT(
  studentID    PK,
  fullName,
  dateOfBirth,
  enrollmentDate
)

INSTRUCTOR(
  studentID    PK, FK -> STUDENT.studentID,
  staffStartDate,
  status       -- ENUM('paid','volunteer')
)

CLASS(
  classID      PK,
  level,
  dayOfWeek,
  timeSlot,
  room,
  headInstructorID  FK -> INSTRUCTOR.studentID
)

CLASS_MEETING(
  meetingID    PK,
  meetingDate,
  classID      FK -> CLASS.classID
)

```

```

ATTENDANCE(
  studentID    PK, FK -> STUDENT.studentID,
  meetingID    PK, FK -> CLASS_MEETING.meetingID
)

MEETING_STAFF(
  meetingID    PK, FK -> CLASS_MEETING.meetingID,
  instructorID PK, FK -> INSTRUCTOR.studentID,
  role         -- ENUM('lead','assistant')
)

CERTIFICATE(
  certID       PK,
  certName     AK (UNIQUE),
  badgeColor,
  description
)

REQUIREMENT(
  reqID        PK,
  description,
  certID       FK -> CERTIFICATE.certID
)

STUDENT_CERTIFICATE(
  studentID    PK, FK -> STUDENT.studentID,
  certID       PK, FK -> CERTIFICATE.certID,
  awardDate
)

```

2.5. Primary key, foreign key, alternate key

Bảng	Primary Key	Foreign Key	Alternate Key / Ghi chú
STUDENT	studentID	—	—
INSTRUCTOR	studentID	studentID → STUDENT.studentID	PK = FK (quan hệ 1–1 với supertype, kiểu "bảng con" cho subtype)
CLASS	classID	headInstructorID → INSTRUCTOR.studentID	AK đề xuất: (level, dayOfWeek, timeSlot, room) — tránh 2 lớp trùng lịch cùng phòng
CLASS_MEETING	meetingID	classID → CLASS.classID	AK đề xuất: (classID, meetingDate) — một lớp không có 2 buổi cùng ngày
ATTENDANCE	(studentID, meetingID)	studentID → STUDENT; meetingID → CLASS_MEETING	—
MEETING_STAFF	(meetingID, instructorID)	meetingID → CLASS_MEETING;	—

Bảng	Primary Key	Foreign Key	Alternate Key / Ghi chú
		instructorID → INSTRUCTOR	
CERTIFICATE	certID	—	certName UNIQUE
REQUIREMENT	reqID	certID → CERTIFICATE.certID	—
STUDENT_CERTIFICATE	(studentID, certID)	studentID → STUDENT; certID → CERTIFICATE	—

2.6. Data dictionary rút gọn

Bảng	Thuộc tính / ràng buộc chính	Giải thích
STUDENT	studentID PK (auto); fullName NOT NULL; dateOfBirth nullable; enrollmentDate NOT NULL DEFAULT CURRENT_DATE	Thông tin định danh học viên khi đăng ký.
INSTRUCTOR	studentID PK/FK; staffStartDate NOT NULL; status NOT NULL ENUM('paid','volunteer')	Vai trò người phụ trách giảng dạy, gán 1–1 với một STUDENT đã tồn tại.
CLASS	classID PK; level NOT NULL; dayOfWeek NOT NULL; timeSlot NOT NULL; room NOT NULL; headInstructorID NOT NULL FK	Lớp học theo lịch cố định, luôn có người phụ trách chính.
CLASS_MEETING	meetingID PK; meetingDate NOT NULL; classID NOT NULL FK	Một buổi học cụ thể của một lớp cố định.
ATTENDANCE	PK(studentID, meetingID); 2 FK NOT NULL	Bảng kết hợp Student – ClassMeeting (điểm danh).
MEETING_STAFF	PK(meetingID, instructorID); role NOT NULL ENUM('lead','assistant')	Bảng kết hợp ClassMeeting – Instructor, ghi vai trò từng buổi.
CERTIFICATE	certID PK; certName UNIQUE NOT NULL; badgeColor NOT NULL; description NULL	Danh mục chứng chỉ nội bộ.
REQUIREMENT	reqID PK; description NOT NULL; certID NOT NULL FK	Yêu cầu cụ thể để đạt một chứng chỉ.
STUDENT_CERTIFICATE	PK(studentID, certID); awardDate NOT NULL	Lịch sử chứng chỉ học viên đã đạt được.

2.7. Mapping cần giải thích

- Quan hệ 1–N (CLASS→CLASS_MEETING, INSTRUCTOR→CLASS, CERTIFICATE→REQUIREMENT) được map bằng FK đặt ở bảng phía N (CLASS_MEETING.classID, CLASS.headInstructorID, REQUIREMENT.certID).

- Ba quan hệ N–M có thuộc tính riêng (ATTENDANCE, MEETING_STAFF, STUDENT_CERTIFICATE) được map thành ba bảng trung gian độc lập, mỗi bảng có PK phức hợp gồm 2 FK, đúng theo associative entity đã xác định ở ERD.
- Subtype INSTRUCTOR của STUDENT được map theo phương án “bảng con dùng chung khóa” (one-table-per-subtype, shared PK/FK): INSTRUCTOR.studentID vừa là PK vừa là FK trỏ về STUDENT.studentID. Phương án này được chọn thay vì gộp chung một bảng STUDENT có các cột staffStartDate/status luôn NULL cho học viên thường, giúp tránh NULL thừa (theo khuyến nghị Mục 3.2/9 của tài liệu hướng dẫn).
- Không có thuộc tính đa trị hay repeating group nào trong thiết kế (ví dụ không dùng Requirement1, Requirement2... trong CERTIFICATE); mọi danh sách được tách thành bảng con tương ứng.
- Business identifier certName của CERTIFICATE được đánh dấu UNIQUE vì đề bài ngụ ý mỗi chứng chỉ có một tên xác định (ví dụ 'Starter Badge').

2.8. Relation ban đầu để phân tích

Để minh họa quá trình chuẩn hóa, nhóm dựng một relation gộp giả định (unnormalized/1NF ban đầu) mô phỏng dữ liệu như khi trung tâm ghi chép thủ công điểm danh — mọi thông tin nằm trên một dòng. Sau đó áp dụng phân tích phụ thuộc hàm để suy ra các relation cuối cùng ở Mục 2.4, cho thấy chúng đạt tối thiểu 3NF.

```
ATTENDANCE_RECORD(
  studentID, studentFullName, studentDOB, studentEnrollDate,
  meetingID, meetingDate,
  classID, classLevel, classDayOfWeek, classTimeSlot, classRoom,
  headInstructorID, headInstructorName, headInstructorStatus,
  certID, certName, certBadgeColor, awardDate
)
```

Candidate key ứng viên: (studentID, meetingID, certID)

-- nhưng đây không phải một khóa hợp lệ về mặt ngữ nghĩa (certID không phụ thuộc
 -- vào việc điểm danh một buổi cụ thể) → chính dấu hiệu này cho thấy relation
 -- gộp đang vi phạm chuẩn hóa và cần được tách.

2.9. Các phụ thuộc hàm (Functional Dependencies)

Mã	Functional Dependency	Hệ quả
FD1	studentID → studentFullName, studentDOB, studentEnrollDate	Thông tin học viên không nên lặp lại theo mỗi buổi điểm danh hay mỗi chứng chỉ → tách STUDENT.
FD2	meetingID → meetingDate, classID	Thông tin buổi học chỉ phụ thuộc meetingID → tách CLASS_MEETING.
FD3	classID → classLevel, classDayOfWeek, classTimeSlot, classRoom, headInstructorID	Thông tin lớp học chỉ phụ thuộc classID, không phụ thuộc meetingID → tách CLASS; đây là transitive dependency qua classID (meetingID → classID → classLevel...) nếu giữ nguyên relation gộp.
FD4	headInstructorID → headInstructorName, headInstructorStatus	Thông tin instructor phụ thuộc headInstructorID, không phụ thuộc classID trực tiếp về mặt ngữ nghĩa

Mã	Functional Dependency	Hệ quả
		(transitive qua classID) → tách INSTRUCTOR (và STUDENT).
FD5	certID → certName, certBadgeColor	Thông tin chứng chỉ chỉ phụ thuộc certID → tách CERTIFICATE.
FD6	(studentID, meetingID) → (không có thuộc tính điểm danh bổ sung ngoài khóa trong ví dụ)	Xác nhận ATTENDANCE là bảng kết hợp thuần khóa ngoại.
FD7	(studentID, certID) → awardDate	awardDate phụ thuộc đầy đủ vào cặp khóa phức hợp (studentID, certID), không phụ thuộc bộ phận → xác nhận STUDENT_CERTIFICATE đạt 2NF.

2.10. Phân tích 1NF, 2NF, 3NF và decomposition

Mức	Cần chứng minh	Áp dụng trong thiết kế BrightPath
1NF	Mỗi thuộc tính là atomic; không có repeating group / thuộc tính đa trị.	Không lưu danh sách buổi học hay danh sách chứng chỉ trong một cột của STUDENT; mỗi buổi điểm danh, mỗi chứng chỉ đạt được là một dòng riêng trong ATTENDANCE / STUDENT_CERTIFICATE.
2NF	Không có partial dependency của thuộc tính không khóa lên một phần của khóa phức hợp.	ATTENDANCE và MEETING_STAFF chỉ chứa các cột khóa (không có thuộc tính mô tả nào phụ thuộc bộ phận); STUDENT_CERTIFICATE.awardDate phụ thuộc đầy đủ vào cặp (studentID, certID) theo FD7, không phụ thuộc riêng studentID hay riêng certID.
3NF	Không có transitive dependency của thuộc tính không khóa qua một thuộc tính không khóa khác.	Trong relation gộp ban đầu, headInstructorName phụ thuộc headInstructorID, mà headInstructorID lại phụ thuộc classID (transitive qua FD3+FD4) — sau khi tách CLASS và INSTRUCTOR/STUDENT riêng, mỗi thuộc tính không khóa trong từng bảng con chỉ còn phụ thuộc trực tiếp vào khóa chính của bảng đó.

Kết luận chuẩn hóa

Dựa trên FD1–FD7, relation gộp ATTENDANCE_RECORD được tách losslessly (không mất thông tin, có thể join lại đúng qua các FK) thành 9 relations cuối cùng: STUDENT, INSTRUCTOR, CLASS, CLASS_MEETING, CERTIFICATE, REQUIREMENT, ATTENDANCE, MEETING_STAFF, STUDENT_CERTIFICATE. Trong mỗi relation, mọi thuộc tính không khóa đều phụ thuộc vào toàn bộ khóa chính và không phụ thuộc bắc cầu qua thuộc tính không khóa khác — do đó các relation đạt tối thiểu 3NF.

2.11. Ghi chú về denormalization

Thiết kế hiện tại không áp dụng denormalization có chủ đích nào. Nếu về sau phát sinh nhu cầu báo cáo tần suất cao (ví dụ: đếm nhanh số buổi một học viên đã tham dự trong tháng), nhóm đề xuất dùng view tổng hợp (SELECT ... GROUP BY) hoặc bảng tổng hợp định kỳ (summary table) được cập nhật bằng batch job, thay vì phi chuẩn hóa trực tiếp các bảng giao dịch (STUDENT, ATTENDANCE) — giữ nguyên tắc 3NF cho lớp dữ liệu giao dịch.

3. TRIỂN KHAI SCHEMA VÀ DỮ LIỆU

3.1. Quy ước đặt tên và lựa chọn kiểu dữ liệu

- Tên bảng/cột dùng snake_case ở physical model (ví dụ studentID → student_id), thống nhất với data dictionary ở Mục 2.6.
- Khóa chính surrogate (AUTO_INCREMENT) dùng cho các entity thông thường (student, class, class_meeting, certificate, requirement) vì đề bài không cho một mã nghiệp vụ tự nhiên ổn định.
- Khóa chính của bảng subtype (instructor) và các bảng trung gian dùng khóa phức hợp/dùng chung, không phát sinh surrogate riêng, để phản ánh đúng bản chất 1–1 hoặc N–M.
- ENUM được dùng cho các thuộc tính có miền giá trị rõ ràng, hữu hạn theo đề bài: instructor.status ('paid','volunteer'), meeting_staff.role ('lead','assistant').
- DATE dùng cho ngày (date_of_birth, enrollment_date, meeting_date, staff_start_date, award_date); không cần thành phần giờ.

3.2. Thứ tự tạo bảng

Bảng cha (không phụ thuộc FK) được tạo trước, bảng con (có FK) tạo sau, đảm bảo script chạy được trên schema rỗng theo đúng thứ tự phụ thuộc: student → instructor → certificate → class → class_meeting → requirement → attendance → meeting_staff → student_certificate.

3.3. CREATE DATABASE / CREATE TABLE

```
DROP DATABASE IF EXISTS brightpath_db;
CREATE DATABASE brightpath_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE brightpath_db;

-- =====
-- 1) STUDENT — bảng gốc, mọi học viên khi đăng ký
-- =====

CREATE TABLE student (
  student_id  BIGINT NOT NULL AUTO_INCREMENT,
  full_name   VARCHAR(120) NOT NULL,
  date_of_birth DATE NULL,
  enrollment_date DATE NOT NULL DEFAULT (CURRENT_DATE),
  PRIMARY KEY (student_id)
) ENGINE = InnoDB;
```

```
-- =====
-- 2) INSTRUCTOR — subtype (partial) của STUDENT, khóa dùng chung
-- =====
```

```
CREATE TABLE instructor (
  student_id BIGINT NOT NULL,
  staff_start_date DATE NOT NULL,
  status ENUM('paid', 'volunteer') NOT NULL,
  PRIMARY KEY (student_id),
  CONSTRAINT fk_instructor_student
    FOREIGN KEY (student_id)
      REFERENCES student(student_id)
    ON UPDATE RESTRICT
    ON DELETE RESTRICT
) ENGINE = InnoDB;
```

```
-- =====
-- 3) CERTIFICATE — danh mục chứng chỉ nội bộ
-- =====
```

```
CREATE TABLE certificate (
  cert_id INT NOT NULL AUTO_INCREMENT,
  cert_name VARCHAR(100) NOT NULL,
  badge_color VARCHAR(30) NOT NULL,
  description VARCHAR(500) NULL,
  PRIMARY KEY (cert_id),
  CONSTRAINT uq_certificate_name UNIQUE (cert_name)
) ENGINE = InnoDB;
```

```
-- =====
-- 4) CLASS — lớp học theo lịch cố định; FK bắt buộc tới INSTRUCTOR
-- =====
```

```
CREATE TABLE class (
  class_id INT NOT NULL AUTO_INCREMENT,
  level VARCHAR(60) NOT NULL,
  day_of_week ENUM('MON','TUE','WED','THU','FRI','SAT','SUN') NOT NULL,
  time_slot TIME NOT NULL,
  room VARCHAR(20) NOT NULL,
  head_instructor_id BIGINT NOT NULL,
  PRIMARY KEY (class_id),
  CONSTRAINT uq_class_schedule
    UNIQUE (day_of_week, time_slot, room),
  CONSTRAINT fk_class_head_instructor
    FOREIGN KEY (head_instructor_id)
      REFERENCES instructor(student_id)
    ON UPDATE RESTRICT
    ON DELETE RESTRICT
) ENGINE = InnoDB;
```

Giải thích lựa chọn (Mục 3.3)

- `uq_class_schedule (day_of_week, time_slot, room)`: ngăn hai lớp khác nhau bị xếp trùng phòng, trùng thời điểm — một ràng buộc hợp lý suy ra từ ví dụ trong đề bài dù không được phát biểu tường minh.

- FK head_instructor_id NOT NULL + RESTRICT: đảm bảo mọi CLASS luôn có đúng một người phụ trách chính (BR04); không cho xóa INSTRUCTOR đang là head của lớp nào — buộc phải chuyển giao lớp trước khi xóa/vô hiệu hóa instructor.
- instructor.student_id vừa là PK vừa là FK: hiện thực subtype theo phương án shared-key, đảm bảo mỗi instructor ứng với đúng một student đã tồn tại.

3.4. CREATE TABLE — bảng giao dịch và bảng kết hợp

```
-- =====
-- 5) CLASS_MEETING — buổi học cụ thể của một CLASS
-- =====
```

```
CREATE TABLE class_meeting (
  meeting_id BIGINT NOT NULL AUTO_INCREMENT,
  meeting_date DATE NOT NULL,
  class_id INT NOT NULL,
  PRIMARY KEY (meeting_id),
  CONSTRAINT uq_meeting_natural_key
    UNIQUE (class_id, meeting_date),
  CONSTRAINT fk_meeting_class
    FOREIGN KEY (class_id)
    REFERENCES class(class_id)
    ON UPDATE RESTRICT
    ON DELETE RESTRICT
) ENGINE = InnoDB;
```

```
-- =====
-- 6) REQUIREMENT — yêu cầu của một CERTIFICATE
-- =====
```

```
CREATE TABLE requirement (
  req_id INT NOT NULL AUTO_INCREMENT,
  description VARCHAR(300) NOT NULL,
  cert_id INT NOT NULL,
  PRIMARY KEY (req_id),
  CONSTRAINT fk_requirement_certificate
    FOREIGN KEY (cert_id)
    REFERENCES certificate(cert_id)
    ON UPDATE RESTRICT
    ON DELETE CASCADE
) ENGINE = InnoDB;
```

```
-- =====
-- 7) ATTENDANCE — bảng kết hợp STUDENT – CLASS_MEETING (N–M)
-- =====
```

```
CREATE TABLE attendance (
  student_id BIGINT NOT NULL,
  meeting_id BIGINT NOT NULL,
  PRIMARY KEY (student_id, meeting_id),
  CONSTRAINT fk_attendance_student
    FOREIGN KEY (student_id)
    REFERENCES student(student_id)
    ON UPDATE RESTRICT
    ON DELETE RESTRICT,
  CONSTRAINT fk_attendance_meeting
```

```

FOREIGN KEY (meeting_id)
REFERENCES class_meeting(meeting_id)
ON UPDATE RESTRICT
ON DELETE CASCADE
) ENGINE = InnoDB;

-- =====
-- 8) MEETING_STAFF — bảng kết hợp CLASS_MEETING – INSTRUCTOR (N–M)
-- =====

CREATE TABLE meeting_staff (
meeting_id BIGINT NOT NULL,
instructor_id BIGINT NOT NULL,
role ENUM('lead', 'assistant') NOT NULL,
PRIMARY KEY (meeting_id, instructor_id),
CONSTRAINT fk_staff_meeting
FOREIGN KEY (meeting_id)
REFERENCES class_meeting(meeting_id)
ON UPDATE RESTRICT
ON DELETE CASCADE,
CONSTRAINT fk_staff_instructor
FOREIGN KEY (instructor_id)
REFERENCES instructor(student_id)
ON UPDATE RESTRICT
ON DELETE RESTRICT
) ENGINE = InnoDB;

-- =====
-- 9) STUDENT_CERTIFICATE — bảng kết hợp STUDENT – CERTIFICATE (N–M)
-- =====

CREATE TABLE student_certificate (
student_id BIGINT NOT NULL,
cert_id INT NOT NULL,
award_date DATE NOT NULL,
PRIMARY KEY (student_id, cert_id),
CONSTRAINT fk_studentcert_student
FOREIGN KEY (student_id)
REFERENCES student(student_id)
ON UPDATE RESTRICT
ON DELETE RESTRICT,
CONSTRAINT fk_studentcert_certificate
FOREIGN KEY (cert_id)
REFERENCES certificate(cert_id)
ON UPDATE RESTRICT
ON DELETE RESTRICT
) ENGINE = InnoDB;

-- =====
-- 10) ATTENDANCE_AUDIT — bảng lưu vết thao tác xóa điểm danh
-- =====

CREATE TABLE attendance_audit (
audit_id BIGINT NOT NULL AUTO_INCREMENT,
student_id BIGINT NOT NULL,
meeting_id BIGINT NOT NULL,
action ENUM('DELETE') NOT NULL,
changed_by VARCHAR(100) NOT NULL,

```

```
changed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (audit_id)
) ENGINE = InnoDB;
```

Giải thích ON DELETE / ON UPDATE

- RESTRICT cho các bảng danh mục/master (student, certificate, instructor, class): tránh xóa dữ liệu gốc khi còn bản ghi phụ thuộc, buộc quy trình nghiệp vụ rõ ràng (vô hiệu hóa thay vì xóa cứng).
- CASCADE cho requirement (theo certificate) và attendance/meeting_staff (theo class_meeting): nếu một buổi học hoặc chứng chỉ bị xóa (trường hợp hiếm, ví dụ nhập sai), các dòng điểm danh/phân công/yêu cầu con phát sinh từ nó cũng nên được dọn theo, vì chúng không có ý nghĩa tồn tại độc lập.

3.5. DỮ LIỆU MẪU (Seed Data)

Dữ liệu mẫu được cấp phát từ file 02_seed_data.sql, khởi tạo đầy đủ các bảng danh mục (Certificate), bảng Master (Student, Class) và dữ liệu giao dịch (Class_Meeting, Attendance, Meeting_Staff) nhằm đáp ứng dữ liệu để kiểm thử toàn bộ Query Pack.

```
-- =====
-- Dữ liệu mẫu cho hệ thống BrightPath Language Center
-- =====

USE brightpath_db;

-- =====
-- Thêm chứng chỉ
-- =====
INSERT INTO certificate (cert_name, badge_color, description) VALUES
('Starter Badge', 'bronze', 'Chứng chỉ khởi đầu, cấp tự động khi đăng ký học.'),
('Fluent Speaker', 'gold', 'Chứng nhận khả năng giao tiếp trôi chảy.'),
('Grammar Master', 'silver', 'Chứng nhận nắm vững ngữ pháp nâng cao.');
```

```
-- =====
-- Thêm yêu cầu của chứng chỉ
-- =====
INSERT INTO requirement (description, cert_id) VALUES
('Hoàn thành tối thiểu 30 buổi học có điểm danh', 2),
('Đạt bài kiểm tra nói cuối cấp độ Advanced', 2),
('Đạt bài kiểm tra ngữ pháp cấp độ Intermediate trở lên', 3);

-- =====
-- Thêm học viên
-- =====
INSERT INTO student (full_name, date_of_birth, enrollment_date) VALUES
('Nguyen Thi Lan', '2001-04-12', '2023-01-10'),
```

```
( 'Tran Van Minh', '1999-09-02', '2022-05-20'),
( 'Le Hoang Nam', '2006-11-30', '2025-02-15'),
( 'Pham Thi Hoa', '2007-03-08', '2026-06-01'),
( 'Do Anh Tuan', '2002-07-19', '2023-08-01'),
( 'Vu Thi Mai', '2005-01-25', '2024-03-11'),
( 'Bui Van Khoa', '2008-05-05', '2025-09-01'),
( 'Ngo Thi Yen', '2003-12-14', '2024-11-20');
```

```
-- =====
-- Thêm giảng viên/trợ giảng
-- =====
```

```
INSERT INTO instructor (student_id, staff_start_date, status) VALUES
(1, '2023-06-01', 'paid'),
(2, '2024-01-15', 'volunteer'),
(5, '2023-09-01', 'paid');
```

```
-- =====
-- Thêm lớp học
-- =====
```

```
INSERT INTO class (level, day_of_week, time_slot, room, head_instructor_id) VALUES
( 'Intermediate English', 'MON', '18:00:00', 'A1', 1),
( 'Beginner Korean', 'TUE', '17:30:00', 'B2', 1),
( 'Advanced English', 'WED', '19:00:00', 'A2', 5),
( 'Beginner Japanese', 'THU', '18:30:00', 'C1', 5);
```

```
-- =====
-- Thêm buổi học
-- =====
```

```
INSERT INTO class_meeting (meeting_date, class_id) VALUES
('2026-06-08', 1), ('2026-06-15', 1), ('2026-06-22', 1), ('2026-06-29', 1),
('2026-06-09', 2), ('2026-06-16', 2),
('2026-06-10', 3), ('2026-06-17', 3), ('2026-06-24', 3),
('2026-06-11', 4);
```

```
-- =====
-- Thêm người phụ trách từng buổi học
-- =====
```

```
INSERT INTO meeting_staff (meeting_id, instructor_id, role) VALUES
(1, 1, 'lead'), (1, 2, 'assistant'),
(2, 1, 'lead'),
(3, 1, 'lead'), (3, 2, 'assistant'),
(4, 1, 'lead'),
(5, 1, 'lead'),
(6, 1, 'lead'),
(7, 5, 'lead'),
(8, 5, 'lead'),
(9, 5, 'lead'),
(10, 5, 'lead');
```

```
-- =====
-- Thêm điểm danh
-- =====
```

```
INSERT INTO attendance (student_id, meeting_id) VALUES
```

```
(3, 1), (4, 1), (6, 1),
(3, 2), (4, 2),
(3, 3), (4, 3), (6, 3),
(3, 4),
(7, 5), (8, 5),
(7, 6),
(6, 7), (8, 7),
(6, 8),
(6, 9);
```

```
-- =====
-- Thêm lịch sử chứng chỉ học viên
-- =====
```

```
INSERT INTO student_certificate (student_id, cert_id, award_date) VALUES
```

```
(3, 2, '2026-06-30'),
(6, 3, '2026-06-25');
```

4. SQL QUERY PACK

Hệ thống sử dụng 8 câu truy vấn để giải quyết nghiệp vụ:

```
-- =====
-- Q01: Lọc và sắp xếp danh sách lớp học (Toán tử LIKE)
-- =====
```

```
SELECT
    class_id,
    level,
    day_of_week,
    time_slot,
    room
FROM class
WHERE level LIKE 'Advanced%'
ORDER BY day_of_week, time_slot;
```

```
-- =====
-- Q02: Báo cáo chi tiết điểm danh (INNER JOIN 4 bảng)
-- =====
```

```
SELECT
    s.full_name,
    c.level,
    cm.meeting_date
FROM attendance AS a
JOIN student AS s
    ON s.student_id = a.student_id
JOIN class_meeting AS cm
```

```

    ON cm.meeting_id = a.meeting_id
JOIN class AS c
    ON c.class_id = cm.class_id
ORDER BY s.full_name, cm.meeting_date;

```

```

-- =====
-- Q03: Kiểm tra buổi học bị bỏ sót điểm danh (LEFT JOIN)
-- =====

```

```

SELECT
    cm.meeting_id,
    cm.meeting_date,
    c.level
FROM class_meeting AS cm
JOIN class AS c
    ON c.class_id = cm.class_id
LEFT JOIN attendance AS a
    ON a.meeting_id = cm.meeting_id
WHERE a.meeting_id IS NULL
ORDER BY cm.meeting_date;

```

```

-- =====
-- Q04: Tìm học viên lười học (GROUP BY & HAVING)
-- =====

```

```

SELECT
    s.student_id,
    s.full_name,
    COUNT(a.meeting_id) AS attendance_count
FROM student AS s
LEFT JOIN attendance AS a
    ON a.student_id = s.student_id
GROUP BY s.student_id, s.full_name
HAVING COUNT(a.meeting_id) < 3
ORDER BY attendance_count ASC;

```

```

-- =====
-- Q05: Học viên chưa có thành tích (Subquery với NOT EXISTS)
-- =====

```

```

SELECT
    s.student_id,
    s.full_name
FROM student AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM student_certificate AS sc
    JOIN certificate AS cert
        ON cert.cert_id = sc.cert_id
    WHERE sc.student_id = s.student_id
        AND cert.cert_name <> 'Starter Badge'
)

```

```
ORDER BY s.full_name;
```

```
-- =====
-- Q06: Bảng xếp hạng học viên chuyên cần (CTE & Window Function)
-- =====
```

```
WITH attendance_count AS (
    SELECT
        student_id,
        COUNT(*) AS total_meetings
    FROM attendance
    GROUP BY student_id
)
SELECT
    s.full_name,
    ac.total_meetings,
    RANK() OVER (ORDER BY ac.total_meetings DESC) AS rank_in_center
FROM attendance_count AS ac
JOIN student AS s
    ON s.student_id = ac.student_id
ORDER BY ac.total_meetings DESC;
```

```
-- =====
-- Q07: Thống kê số lượng đi học theo tuần (Date Function)
-- =====
```

```
SELECT
    YEARWEEK(cm.meeting_date, 3) AS year_week,
    COUNT(a.student_id) AS total_attendance
FROM class_meeting AS cm
LEFT JOIN attendance AS a
    ON a.meeting_id = cm.meeting_id
GROUP BY YEARWEEK(cm.meeting_date, 3)
ORDER BY year_week;
```

```
-- =====
-- Q08: Hiệu suất giảng viên (Truy xuất từ View & Function)
-- =====
```

```
SELECT
    v.level,
    v.total_meetings,
    v.avg_attendance_per_meeting
FROM vw_class_attendance_summary AS v
ORDER BY v.avg_attendance_per_meeting DESC;
```

5. VIEWS VÀ STORED PROGRAMS

5.1. Hệ thống Báo cáo bằng View

```
-- =====
-- 5.1. Hệ thống Báo cáo bằng View
```

```

-- =====
-- 1. View: vw_instructor_workload
-- Thống kê khối lượng công việc của từng giảng viên (lead/assistant)
-- =====

CREATE OR REPLACE VIEW vw_instructor_workload AS
SELECT
    i.student_id AS instructor_id,
    s.full_name,
    i.status,
    COUNT(DISTINCT c.class_id) AS classes_as_head,
    COALESCE(SUM(CASE WHEN ms.role = 'lead' THEN 1 ELSE 0 END), 0) AS meetings_as_lead,
    COALESCE(SUM(CASE WHEN ms.role = 'assistant' THEN 1 ELSE 0 END), 0) AS
meetings_as_assistant
FROM instructor AS i
JOIN student AS s
    ON s.student_id = i.student_id
LEFT JOIN class AS c
    ON c.head_instructor_id = i.student_id
LEFT JOIN meeting_staff AS ms
    ON ms.instructor_id = i.student_id
GROUP BY i.student_id, s.full_name, i.status;

-- =====

-- 2. View: vw_class_attendance_summary
-- Thống kê sĩ số trung bình mỗi buổi của từng lớp
-- =====

CREATE OR REPLACE VIEW vw_class_attendance_summary AS
SELECT
    c.class_id,
    c.level,
    c.day_of_week,
    COUNT(DISTINCT cm.meeting_id) AS total_meetings,
    COUNT(a.student_id) AS total_attendances,
    IF(COUNT(DISTINCT cm.meeting_id) = 0, 0,
        ROUND(COUNT(a.student_id) / COUNT(DISTINCT cm.meeting_id), 2)) AS
avg_attendance_per_meeting
FROM class AS c
LEFT JOIN class_meeting AS cm
    ON cm.class_id = c.class_id
LEFT JOIN attendance AS a
    ON a.meeting_id = cm.meeting_id
GROUP BY c.class_id, c.level, c.day_of_week;

```

5.2. Function đếm số buổi học

```

-- =====
-- 5.2. Function đếm số buổi học
-- =====

CREATE FUNCTION fn_student_attendance_count(p_student_id BIGINT)
RETURNS INT

```

```

READS SQL DATA DETERMINISTIC
BEGIN
    DECLARE v_count INT;

    SELECT COUNT(*) INTO v_count
    FROM attendance
    WHERE student_id = p_student_id;

    RETURN v_count;
END$$

```

5.3. Procedure cấp chứng chỉ (Áp dụng TRANSACTION)

```

-- =====
-- 5.3. Procedure cấp chứng chỉ (Áp dụng TRANSACTION)
-- =====
DELIMITER $$
CREATE PROCEDURE sp_award_certificate(
    IN p_student_id BIGINT,
    IN p_cert_id INT,
    IN p_award_date DATE
)
BEGIN
    DECLARE v_attendance_count INT DEFAULT 0;

    -- Kiểm tra cho chứng chỉ Fluent Speaker (cert_id = 2)
    IF p_cert_id = 2 THEN
        SET v_attendance_count = fn_student_attendance_count(p_student_id);
        IF v_attendance_count < 30 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Học viên chưa đủ số buổi tối thiểu cho Fluent Speaker.';
        END IF;
    END IF;

    START TRANSACTION;
    INSERT INTO student_certificate (student_id, cert_id, award_date)
    VALUES (p_student_id, p_cert_id, p_award_date);
    COMMIT;
END$$
DELIMITER ;

```

6. TRIGGER VÀ EVENT THEO PHẠM VI

Trigger cho các rule không thể enforce bằng CREATE TABLE cơ bản

BR12 — Tự động cấp Starter Badge cho học viên mới: khi một STUDENT được thêm, hệ thống tự động thêm một dòng STUDENT_CERTIFICATE ứng với chứng chỉ có tên 'Starter Badge'.

```

DELIMITER $$

```

```

CREATE TRIGGER trg_student_after_insert
AFTER INSERT ON student
FOR EACH ROW
BEGIN
    DECLARE starter_cert_id INT;

    SELECT cert_id INTO starter_cert_id
    FROM certificate
    WHERE cert_name = 'Starter Badge'
    LIMIT 1;

    IF starter_cert_id IS NOT NULL THEN
        INSERT INTO student_certificate (student_id, cert_id, award_date)
        VALUES (NEW.student_id, starter_cert_id, CURDATE());
    END IF;
END$$

DELIMITER ;

```

BR04 — Ràng buộc “mỗi buổi học phải có đúng một head instructor”: CREATE TABLE cơ bản chỉ đảm bảo mỗi CLASS có đúng 1 head_instructor_id (FK NOT NULL). Riêng ràng buộc “mỗi buổi học (meeting) có đúng một người vai trò lead trong MEETING_STAFF” không thể biểu diễn bằng CHECK constraint tiêu chuẩn của MySQL (CHECK không so sánh được giữa các dòng khác nhau); nhóm đề xuất trigger kiểm tra tại thời điểm ghi:

```

DELIMITER $$

CREATE TRIGGER trg_meeting_staff_before_insert
BEFORE INSERT ON meeting_staff
FOR EACH ROW
BEGIN
    DECLARE lead_count INT;

    IF NEW.role = 'lead' THEN
        SELECT COUNT(*) INTO lead_count
        FROM meeting_staff
        WHERE meeting_id = NEW.meeting_id
        AND role = 'lead';

        IF lead_count >= 1 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Mỗi buổi học chỉ được có đúng một lead instructor.';
        END IF;
    END IF;
END$$

DELIMITER ;

```

BR13 và BR15 — không enforce bằng trigger đơn giản

Hai ràng buộc sau được xử lý ở tầng ứng dụng/quy trình vận hành thay vì trigger CSDL, vì lý do trình bày ở Chương 9:

- BR13 (mọi CERTIFICATE phải có ≥ 1 STUDENT_CERTIFICATE): không thể kiểm tra tại thời điểm INSERT certificate (vì lúc đó chưa có học viên nào đạt được) — đây là ràng buộc cần được đảm bảo qua quy trình vận hành (mỗi chứng chỉ mới phải có kế hoạch/lộ trình được ít nhất một học viên đạt trong thời gian hợp lý), không phải ràng buộc tức thời.
- BR15 (Starter Badge có thể 0 yêu cầu, các chứng chỉ khác phải có ≥ 1 yêu cầu): tương tự, không thể enforce tại thời điểm tạo CERTIFICATE (vì requirement được thêm sau); nên kiểm tra ở tầng ứng dụng trước khi “kích hoạt” một chứng chỉ mới cho học viên đăng ký.

```
-- =====
-- Trigger trg_attendance_after_delete (Ghi log khi xóa điểm danh)
-- =====
CREATE TRIGGER trg_attendance_after_delete
AFTER DELETE ON attendance
FOR EACH ROW
BEGIN
    INSERT INTO attendance_audit (student_id, meeting_id, action, changed_by)
    VALUES (OLD.student_id, OLD.meeting_id, 'DELETE', CURRENT_USER());
END;

-- =====
-- Event ev_cleanup_old_attendance_audit (Dọn log audit cũ)
-- =====
CREATE EVENT ev_cleanup_old_attendance_audit
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP + INTERVAL 1 DAY
ON COMPLETION PRESERVE
DISABLE
DO
    DELETE FROM attendance_audit
    WHERE changed_at < CURRENT_TIMESTAMP - INTERVAL 180 DAY;
```

7. INDEX VÀ EXPLAIN

7.1. Indexes và query mẫu

Ngoài các UNIQUE index đã khai báo cùng constraint (đóng vai trò index truy vấn nhanh theo business identifier), nhóm bổ sung các secondary index sau, được lý giải bằng workload truy vấn thường gặp của trung tâm:

```
-- Truy vấn thường gặp 1: lịch sử điểm danh của một học viên, sắp theo ngày gần nhất
CREATE INDEX idx_attendance_student
ON attendance (student_id);

-- Workload: liệt kê các buổi một học viên đã tham dự
-- SELECT cm.meeting_date, c.level
-- FROM attendance a
-- JOIN class_meeting cm ON cm.meeting_id = a.meeting_id
```

```
-- JOIN class c ON c.class_id = cm.class_id
-- WHERE a.student_id = ?
-- ORDER BY cm.meeting_date DESC;

-- Truy vấn thường gặp 2: tra danh sách buổi học theo lớp, sắp theo ngày
CREATE INDEX idx_meeting_class_date
  ON class_meeting (class_id, meeting_date);

-- Truy vấn thường gặp 3: tra các lớp một instructor phụ trách chính
CREATE INDEX idx_class_head_instructor
  ON class (head_instructor_id);
```

Nhóm không tạo index cho mọi cột: ví dụ certificate.description hay class.room không cần index riêng vì hiếm khi dùng làm điều kiện lọc chính (room chỉ tham gia UNIQUE tổ hợp uq_class_schedule, đã có index đi kèm). Việc thêm index có chi phí ghi (write overhead) nên chỉ tạo khi có workload đọc rõ ràng tương ứng.

7.2. Explain

```
EXPLAIN SELECT *
FROM attendance
WHERE student_id = 3;
```

- Trước khi có Index: Cột 'type = ALL', MySQL phải Table Scan quét toàn bộ bảng.
- Sau khi có Index: Cột 'type = ref', 'possible_keys = idx_attendance_student'. Số lượng 'rows' ước tính phải quét giảm mạnh xuống còn 1-3. Cột 'Extra' hiện 'Using index', chứng minh tốc độ phản hồi đạt mức tức thời (milliseconds).

8. ADMINISTRATION AND BACKUP

8.1. Phân quyền truy cập (Least Privilege)

```
-- =====
-- 8.1. Phân quyền truy cập (Least Privilege)
-- =====

-- Tạo Role giới hạn quyền, chỉ cho nhân sự quyền ĐỌC dữ liệu.
CREATE ROLE IF NOT EXISTS 'role_course_reporter';
GRANT SELECT ON brightpath_db.* TO 'role_course_reporter';

-- Tạo User và gán Role
CREATE USER IF NOT EXISTS 'course_reporter'@'localhost'
  IDENTIFIED BY 'BrightPathLabPass2026!';
GRANT 'role_course_reporter' TO 'course_reporter'@'localhost';
SET DEFAULT ROLE 'role_course_reporter' TO 'course_reporter'@'localhost';
```

8.2. Kế hoạch Backup/Restore cục bộ

```
-- =====
-- 8.2. Kế hoạch Backup/Restore cục bộ
-- =====

-- Lệnh sao lưu toàn bộ (kèm routines, triggers):
-- mysqldump -u root -p --routines --triggers brightpath_db > brightpath_db_backup.sql

-- Lệnh phục hồi vào schema test (Restore Runbook):
```

```
-- mysql -u root -p brightpath_db_restore_test < brightpath_db_backup.sql
```

9. KIỂM THỬ VÀ KẾT LUẬN

9.1. Sample data

Bộ dữ liệu mẫu tối thiểu dưới đây minh họa các trường hợp nghiệp vụ chính: một instructor trạng thái paid, một instructor volunteer chưa được phân lớp, một lớp có nhiều buổi, một buổi có cả lead lẫn assistant, một chứng chỉ Starter Badge không có yêu cầu, và một chứng chỉ khác có yêu cầu kèm học viên đã đạt được.

```
USE brightpath_db;

-- Chứng chỉ: Starter Badge không có yêu cầu; Fluent Speaker có yêu cầu
INSERT INTO certificate (cert_name, badge_color, description) VALUES
('Starter Badge', 'bronze', 'Chứng chỉ khởi đầu, cấp tự động khi đăng ký học.'),
('Fluent Speaker', 'gold', 'Chứng nhận khả năng giao tiếp trôi chảy.');
```

INSERT INTO requirement (description, cert_id) VALUES

```
('Hoàn thành tối thiểu 30 buổi học có điểm danh', 2),
('Đạt bài kiểm tra nói cuối cấp độ Advanced', 2);
```

-- Học viên (một số sẽ trở thành instructor)

```
INSERT INTO student (full_name, date_of_birth, enrollment_date) VALUES
('Nguyen Thi Lan', '2001-04-12', '2023-01-10'), -- 1: sẽ là instructor paid
('Tran Van Minh', '1999-09-02', '2022-05-20'), -- 2: sẽ là instructor volunteer
('Le Hoang Nam', '2006-11-30', '2025-02-15'), -- 3: học viên thường
('Pham Thi Hoa', '2007-03-08', '2026-06-01'); -- 4: học viên mới
```

-- (Trigger trg_student_after_insert tự động cấp Starter Badge cho cả 4 học viên trên)

```
INSERT INTO instructor (student_id, staff_start_date, status) VALUES
(1, '2023-06-01', 'paid'),
(2, '2024-01-15', 'volunteer'); -- volunteer, hiện chưa phụ trách lớp nào
```

-- Lớp học: mỗi lớp có đúng 1 head instructor

```
INSERT INTO class (level, day_of_week, time_slot, room, head_instructor_id) VALUES
('Intermediate English', 'MON', '18:00:00', 'A1', 1),
('Beginner Korean', 'TUE', '17:30:00', 'B2', 1);
```

-- Buổi học cụ thể của lớp Intermediate English

```
INSERT INTO class_meeting (meeting_date, class_id) VALUES
('2026-07-06', 1),
('2026-07-13', 1);
```

-- Buổi 1 có head instructor (id=1) dạy chính, instructor volunteer (id=2) đến hỗ trợ

```
INSERT INTO meeting_staff (meeting_id, instructor_id, role) VALUES
(1, 1, 'lead'),
(1, 2, 'assistant');
```

-- Điểm danh buổi 1: học viên 3 và 4 tham dự; học viên thường có thể học lớp không cố định

```
INSERT INTO attendance (student_id, meeting_id) VALUES
(3, 1),
(4, 1);
```

```
-- Học viên 3 đạt thêm chứng chỉ Fluent Speaker
INSERT INTO student_certificate (student_id, cert_id, award_date) VALUES
(3, 2, '2026-06-30');
```

9.2. Positive tests

Test	Hành động	Kết quả mong đợi
Thêm học viên mới	INSERT vào student	Trigger tự động thêm dòng student_certificate ứng với Starter Badge (BR12)
Instructor chưa có lớp	SELECT instructor không xuất hiện trong class.head_instructor_id	Trả về instructor_id = 2 (volunteer) — xác nhận BR04 cho phép instructor chưa được phân lớp
Buổi học nhiều người phụ trách	SELECT * FROM meeting_staff WHERE meeting_id = 1	Trả về 2 dòng: 1 lead, 1 assistant (BR10)
Học viên chưa tham dự buổi nào	SELECT student_id FROM student WHERE student_id NOT IN (SELECT student_id FROM attendance)	Trả về student_id = 1, 2 (là instructor, không bắt buộc tham dự với vai trò student) — xác nhận BR09

9.3. Negative tests

Test	Kết quả mong đợi	Rule được kiểm tra
INSERT class không có head_instructor_id (NULL)	Bị từ chối do NOT NULL	BR04 — mỗi lớp phải có đúng 1 head instructor
INSERT thêm meeting_staff role='lead' lần 2 cho cùng meeting_id=1	Bị từ chối do trigger trg_meeting_staff_before_insert (SIGNAL SQLSTATE 45000)	BR10 — mỗi buổi chỉ có đúng 1 lead
INSERT student_certificate trùng (student_id, cert_id) đã tồn tại	Bị từ chối do PRIMARY KEY phức hợp	BR11 — mỗi chứng chỉ chỉ trao 1 lần cho 1 học viên
INSERT class_meeting với class_id không tồn tại	Bị từ chối do FOREIGN KEY	BR07 — mỗi buổi học phải gắn đúng 1 lớp có thật
INSERT 2 lớp cùng day_of_week + time_slot + room	Bị từ chối do UNIQUE (uq_class_schedule)	Ràng buộc suy luận từ ví dụ đề bài — tránh trùng lịch/phòng
DELETE certificate đang có student_certificate tham chiếu	Bị từ chối do FK RESTRICT	Bảo vệ toàn vẹn lịch sử chứng chỉ đã trao (liên quan BR13)
Cố tình gọi CALL sp_award_certificate(4, 2) để cấp chứng chỉ Fluent Speaker cho người học dưới 30 buổi.	Bị từ chối (SIGNAL SQLSTATE 45000: Học viên chưa đủ số buổi). Transaction Rollback an toàn.	Quy tắc nghiệp vụ về điều kiện cấp chứng chỉ.

9.4. Query đối chiếu và kết quả

```
-- Đối chiếu 1: xác nhận cấu trúc bảng đã tạo đúng constraint
SHOW CREATE TABLE class;
SHOW CREATE TABLE meeting_staff;
```

```
-- Đối chiếu 2: danh sách buổi học kèm người phụ trách và vai trò
SELECT cm.meeting_date,
       c.level,
       s.full_name AS instructor_name,
       ms.role
FROM meeting_staff AS ms
JOIN class_meeting AS cm ON cm.meeting_id = ms.meeting_id
JOIN class AS c      ON c.class_id = cm.class_id
JOIN instructor AS i  ON i.student_id = ms.instructor_id
JOIN student AS s     ON s.student_id = i.student_id
ORDER BY cm.meeting_date, ms.role;

-- Đối chiếu 3: lịch sử chứng chỉ của từng học viên
SELECT s.full_name, cert.cert_name, sc.award_date
FROM student_certificate AS sc
JOIN student AS s        ON s.student_id = sc.student_id
JOIN certificate AS cert  ON cert.cert_id = sc.cert_id
ORDER BY s.full_name, sc.award_date;

-- Đối chiếu 4: chứng chỉ nào (nếu có) hiện chưa có học viên nào đạt (kiểm tra BR13)
SELECT cert.cert_id, cert.cert_name
FROM certificate AS cert
LEFT JOIN student_certificate AS sc ON sc.cert_id = cert.cert_id
WHERE sc.cert_id IS NULL;
```

Với bộ dữ liệu mẫu ở Chương 9.1, truy vấn đối chiếu 4 trả về tập rỗng — xác nhận mọi chứng chỉ hiện có (Starter Badge, Fluent Speaker) đều đã có ít nhất một học viên đạt được, phù hợp BR13. Khi thêm một CERTIFICATE mới mà chưa có ai đạt, truy vấn này chính là cơ chế kiểm tra ở tầng ứng dụng được đề cập tại Chương 6.

9.5. Kết luận

Báo cáo đã trình bày đầy đủ chuỗi thiết kế cho hệ thống quản lý BrightPath Language Center: xuất phát từ 12 yêu cầu nghiệp vụ gốc, nhóm trích xuất 15 quy tắc nghiệp vụ (BR01–BR15), xây dựng Crow's Foot ERD với 9 thực thể (bao gồm một quan hệ subtype/supertype giữa STUDENT và INSTRUCTOR, và ba associative entity ATTENDANCE, MEETING_STAFF, STUDENT_CERTIFICATE giải quyết các quan hệ N–M có thuộc tính riêng).

Mô hình quan hệ được ánh xạ trực tiếp từ ERD, phân tích phụ thuộc hàm xác nhận thiết kế đạt tối thiểu 3NF mà không cần phi chuẩn hóa. Script MySQL ở Chương 3 và Chương 6 hiện thực đầy đủ PK, FK, UNIQUE, CHECK/ENUM và hai trigger xử lý các quy tắc không thể biểu diễn bằng CREATE TABLE cơ bản (tự động cấp Starter Badge, và giới hạn mỗi buổi học chỉ có một lead instructor).

Hai ràng buộc còn lại — BR13 (mọi chứng chỉ phải từng được trao cho ít nhất một học viên) và BR15 (điều kiện số lượng yêu cầu tùy loại chứng chỉ) — được xác định là ràng buộc tồn tại tối thiểu và ràng buộc điều kiện phức tạp, không thể enforce tại thời điểm tạo bản ghi gốc bằng constraint đơn thuần; báo cáo đề xuất kiểm soát các ràng buộc này qua quy trình vận hành và truy vấn đối chiếu định kỳ ở tầng ứng dụng, đúng theo nguyên tắc “không ngần ngại tuyên bố rằng CREATE TABLE cơ bản đã giải quyết mọi rule”.

Bộ dữ liệu mẫu và các test case dương/âm ở Chương 9 xác nhận thiết kế hoạt động đúng với các tình huống nghiệp vụ tiêu biểu nêu trong đề bài, đồng thời từ chối chính xác các thao tác vi phạm ràng buộc.